⇒ To start project [npm init -y] → default yes

(i) app.js के दो काम → Server of config करना → कौन से middleware, कितने type के API
→ To create instance of server.

⇒ लेकिन server start होता है → server.js file से

⇒ npx → node package execution

Now we have to connect MongoDB

(i) Go to atlas → create a New project → copy link →
Paste in compass → install mongoose in V&C (npm i mongoose),
(npm i dotenv)
इन्हें को .env में store करो → फिर config folder में mongoose से connect
होगा → import in server.js → call function to check

⇒ Now we create a Scheme in Models folder → hashing of Password
→ comparing of Password
using module bcrypt

⇒ Authentication and Authorization →

Folder → Routes → Auth.Routes.(इसमें हम सब routes देंगे) इसके logic
import
app.js
import
(controller folder → Auth.controller)

check user exist(email) → If yes return
NO
Create New user

user से login करते है
फिर JWT token
(Cookie parser)
secret key होनी भी जरूरी Secure
but computational power increase

Mail send करने के लिए → install nodemail
↳ Always refresh token भेज mail
हर से जो test के लिए authorized
फिर बना

user से कहते है लेने
(cookie Cookie i Cookie-Parser)
↳ ऐसे ही register हुए होने पर mail भेजो
वहां अगर चाहो

यहां से third party Service use होगा इसका folder अलग होगा
जैसे email- send करने के लिए google API establish
then install [npm i nodemailer] → then write code
in service

Google hosts it mail server which is called SMTP server जो हमें mail handle
करते है।

हर user के multiple account हो सकता है, तो उसके लिए एक model → account model है → only to hold accounts

Such easy है mongodb से सारे फिर एक relating से use करते

⇒ यहां मैंने /account route पर जा रहा है तो
middleware से check करो कि क्या cookie से और
token आ रहा है or not then return login
उसके बाद user या data के साथ match करेंगे
and then return output
↳ middleware → with middleware → check token
or not when
उसके बाद user के पास का model when

⇒ Now we study how ledger works →
when Person A send money to person B then a transaction
function activate in this function multiple things are present
and then in a idempotency key is also present inside
the function the functionality of idempotency is that
it is generated only one time for one transaction and
that is unique means in every transaction different
type of idempotent key is generated.

हर बार Balance show नहीं करते हैं बल्कि Ledger से जा कर कितना amount credited − amount debited = balance.

⇒ and also status is going pending →
transition {
to → A
From → B
amount → x
Status → Pending
idempotency key → any string type
}

ये होगा client-side से generate होगा ये backend से नहीं

इसका काम कि मान ले कुछ glitch हुआ तो हर transition से कई बार amount जा debited हो जाए इसलिये भी ये key होगा।

Next step →
New ledger is created → It is like a like a register
that at a particular time how much amount is come
and how much amount is goes
In every transaction two ledger entry is happen

entry 1	entry 2

Ledger 1 {
Account − B
amount → x
type → debited (जाया है)
}
	Ledger 2 {
	Account − A
	amount → x
	type → credited (आया है)
	}

when both entry successfull then change
Status : → completed in transaction
There is four steps →

(i) Create a transaction (with status Pending)

(ii) Deducted money from B (with the help of ledger)

(iii) Credited money to A (with the help of ledger)

(iv) change the status → completed

⇒ Ledger control है यहां हम ledger के सारे चीज़ रखते हैं (like statement), कितना amount Person A से deduct करना है इतना amount है कि नहीं to check getbalance() → account. model से भरा है यहां से Aggregation के help से ledger में से a debit- credited

⇒ transactionhelper → में अगर मान लो transaction start हुआ और दो step complete होकर error आ गया तो जो 2 step हुआ है उसका तो entry save हो जाएगा but अगर transaction complete नहीं हुआ है तो inconsistency आ जाएगा तो इससे fight करने के लिए mongodb में startTransaction() दिया है कि इसके बाद या तो complete होगा या rollback हो जाएगा

⇒ जब हम logout हो जाते हैं normal then ना हो कभी भी token generate हुआ है उसके भी Blacklist कर दें,